

Towards a Fully Differentiable Framework for Autonomous Driving Based on Model-Structured Neural Networks

Sabrina Ciuffoletti, Gioele Defrancesco, Giovanni Scialla, Mattia Piazza,
Sebastiano Taddei, Gastone Pietro Rosati Papini

Abstract—Generalization in autonomous driving remains an open challenge, with current approaches divided between model-based pipelines, which ensure interpretability and safety, and end-to-end learning methods, which offer flexibility but limited guarantees. This work proposes a hybrid framework that combines these paradigms through Model Structured Neural Networks (MSNNs) within a fully differentiable architecture. The system integrates vehicle dynamics modeling, control, and motion planning into a unified pipeline. A structured neural forward model is first identified from data, followed by the learning of an inverse controller through a closed-loop, mental simulation approach. A neural planner, based on a modular motion planning structure, is then connected to the control layer, enabling end-to-end training and joint optimization. The majority of the process is supported by nnodely, an open-source framework for the design and training of MSNNs. The paper presents the structure of each module and the integration within the control pipeline. The complete system is validated in closed-loop simulations in IPG CARMAKER®, where the planner network is directly connected to the inverse model to control the vehicle. Results highlight the potential of structured neural approaches for achieving robust and generalizable autonomous driving.

I. INTRODUCTION

Achieving robust task generalization remains one of the central open challenges in autonomous driving and racing [1]. While recent advances in machine learning have enabled impressive perception and control capabilities, current systems still struggle when deployed in unseen environments or under distribution shifts [2]. This limitation has led to the emergence of two dominant and often contrasting paradigms.

On one side, modular architectures grounded in engineering principles and designed for interpretability and safety [3]. On the other, end-to-end learning approaches that leverage large-scale datasets to learn complex behaviors directly from experience [4]. Despite their respective successes, neither paradigm alone has yet demonstrated the ability to reliably generalize across the wide variability of real-world driving conditions [5].

In this context, hybrid approaches that combine model-based and learning methods are gaining increasing attention. Model Structured Neural Network (MSNN) offers a promising direction by embedding prior knowledge directly into

neural architectures (*i.e.*, physical laws and control principles). This integration introduces inductive biases that improve data efficiency, interpretability, and robustness, while retaining the adaptability of learning-based methods [6], [7]. At the same time, a recent work on learning control through mental simulation has shown that it is possible to construct predictive and control models in a fully data-driven yet structured manner [8], by first learning forward dynamics and then synthesizing inverse models through differentiable simulation pipelines. These ideas suggest a pathway toward autonomous systems that are both learnable and controllable, bridging the gap between model-based and data-driven paradigms.

This paper builds on these insights and proposes a unified framework for autonomous driving based on fully differentiable, model-structured neural architectures. The approach integrates three key components, vehicle dynamics modeling, control, and motion planning, within a single end-to-end trainable pipeline. First, a neural forward model of the vehicle dynamics is identified from data using a structured architecture that reflects the underlying physics. Then, an inverse neural controller is synthesized by coupling it with the forward model and training the two in a closed loop, following the mental simulation paradigm. Finally, a high-level planner, inspired by modular motion planning approaches [8], is currently introduced as a model-based module that generates target trajectories, but its structure is designed to be compatible with the nnodely framework¹ to enable future joint training with the control layer through the differentiable vehicle model. Moreover, this hierarchical yet fully differentiable structure enables joint optimization and fine-tuning across all levels of the autonomy stack. Once trained, the control networks are exported to ONNX for deployment. Importantly, their neural nature allows, in principle, for further online adaptation.

A key enabler of this framework is nnodely, an open-source environment specifically designed for the development of MSNNs. By providing modular building blocks aligned with physical modeling and control concepts, nnodely allows the construction of structured neural architectures that can be trained, interconnected, and deployed within a unified workflow. Moreover, the framework supports the composition of multiple neural modules into larger differentiable systems, enabling end-to-end learning across perception, planning, and control layers.

The proposed approach directly addresses the core ques-

The authors are with the Department of Industrial Engineering, University of Trento, Via Sommarive 9, 38123 Povo (TN), Italy (email: gastone.rosatipapini@unitn.it)

This work was supported under the FIS 2 Call, Grant Assignment Decree No. 1236 adopted on 01/08/2023 by the Italian Ministry of University and Research (MUR), for the project FIS-2023-03684 'Structured neural network framework for modeling and control of autonomous systems – Neu4mes', CUP E53C24003800001.

¹Available at <https://github.com/tonegas/nnodely>

tion of this workshop: how to achieve generalization in autonomous driving. By combining structured inductive biases with data-driven learning, it aims to overcome the limitations of both purely modular and purely end-to-end systems. On the one hand, the use of MSNNs provides interpretability, stability, and the possibility of incorporating formal knowledge into the learning process. On the other hand, the differentiable integration of planning and control enables continuous adaptation and improvement from data, including fine-tuning on new scenarios and domains. Ultimately, this work advocates for a hybrid paradigm, where modularity is not opposed to learning, but rather embedded within it. By leveraging structured neural representations and differentiable simulation, the proposed framework moves toward autonomous driving systems that are not only scalable and data-efficient but also robust, interpretable, and capable of generalizing beyond their training distribution.

II. FULLY DIFFERENTIABLE PLANNING ARCHITECTURE

The overall architecture is composed of three main functional blocks: the vehicle model, the low-level control layer, and the motion planner, as illustrated in Fig. 1. At the current stage of development, the vehicle model and the lateral low-level controller are fully implemented and trained using the nnodely framework, which is introduced in Section II-D. In contrast, the motion planner is currently implemented as a model-based module, but its structure has been explicitly designed to be compatible with a future integration within the same framework.

The development of the pipeline follows a staged procedure. First, the vehicle model (Section II-A) is identified from data collected in the IPG CARMAKER[®] simulation environment. Once the forward dynamics are learned, the low-level lateral controller (Section II-B) is trained in closed loop by coupling it with the learned model, following a differentiable training scheme. The same approach can be extended in future work to the longitudinal control layer, as well as to the development of fully coupled lateral-longitudinal models and controllers.

After training, the neural controller is exported in ONNX format through nnodely and integrated into the control pipeline for closed-loop evaluation within IPG CARMAKER[®]. The motion planner (Section II-C), instead, is currently implemented in C++ and interfaced with the control system during the testing phase. However, its bio-inspired and modular structure makes it a suitable candidate for a full re-implementation within nnodely. This would enable the joint optimization of its internal parameters, including both the generation of motion primitives and the computation of their associated salience.

A. Vehicle Model

The vehicle MSNN model for lateral dynamics relates the steering input angle to the resulting vehicle curvature. The network's physical structure is derived from the handling diagram (HD), assuming that the understeer factor K_{us}

varies linearly with lateral acceleration ($a_y = \rho v_x^2$):

$$\rho(a_x, v_x, \delta) = \frac{\delta}{L + K_{us}(a_x)v_x^2}. \quad (1)$$

Here, ρ denotes the vehicle curvature, δ is the steering angle at the wheels, and L represents the wheelbase (inter-axle distance). The proposed neural network predicts the vehicle curvature at the next time step ρ_{k+1} (4) using a time window of $n = 30$ past steering samples, $\delta_k = [\delta_{k-n+1}, \dots, \delta_k]$, together with current longitudinal speed v_k and acceleration a_k . The model is structured as the product of a dynamic term (2) which captures the steering-to-curvature relationship through a speed-dependent *FIR filter* (w_δ are the weights), and a parametric function with parameter A_i (3) that accounts for understeer effects under different longitudinal acceleration conditions. $\phi_{*i}(\cdot)$ denotes triangular fuzzy membership function. The number of channels is $N_v = 8$, $N_a = 5$. The neural model's equations are:

$$D_v(v_k, \delta_k) = \sum_{i=1}^{N_v} \phi_{v_i}(v_k) (w_{\delta_i}^\top \delta_k), \quad (2)$$

$$F_a(a_k, v_k) = \sum_{i=1}^{N_a} \phi_{a_i}(a_k) \frac{1}{1 + A_i v_k^2}, \quad (3)$$

$$\rho_{k+1} = D_v(v_k, \delta_k) F_a(a_k, v_k). \quad (4)$$

B. Lateral Control

The controller receives two future windows of length $n = 30$ for the target velocity v_{t_k} and curvature ρ_{t_k} , along with a past window of realized steering angles $\delta_{c_k} = [\delta_{k-n+1}, \dots, \delta_{k-1}]$ to account for initial steering conditions. A parametric function (6), using parameters A_i learned by the forward model, compensates for understeer effects, while a FIR filter (5) (w_ρ) captures the vehicle's dynamic response. The predicted steering angle $\tilde{\delta}_k$ (7) is obtained as the sum of the target trajectory contribution and the past steering history.

$$K_t(a_k, v_{t_k}, \rho_{t_k}) = \sum_{i=1}^{N_v} \phi_{v_i}(v_k) (w_{\rho_i}^\top \nabla(a_k, v_{t_k}, \rho_{t_k})) \quad (5)$$

$$\nabla(a_k, v_{t_k}, \rho_{t_k}) = \rho_{t_k} \odot \left(1 + \sum_{i=1}^{N_a} \phi_{a_i}(a_k) A_i v_{t_k}^2 \right) \quad (6)$$

$$\tilde{\delta}_k = K_t(a_k, v_k, v_{t_k}, \rho_{t_k}) + w_{\delta_c}^\top \delta_{c_k} \quad (7)$$

C. Motion Planner

The proposed architecture employs a jerk-optimized motion-planning layer to generate smooth, high-fidelity trajectories. At its core, this layer solves a constrained optimization problem to minimize control effort, defined as $J = \int_0^{t_f} \mathbf{u}(t)^2 dt$, where the control input $\mathbf{u}(t)$ corresponds to longitudinal or lateral jerk, similar to [9]. By formulating the motion primitives as the result of an Optimal Control Problem (OCP), the architecture can produce solutions that are inherently compliant with the system dynamics and boundary conditions.

While these primitives can be derived through analytical optimality conditions, the framework is designed to be independent of the underlying solver, as in [10], [11]. This

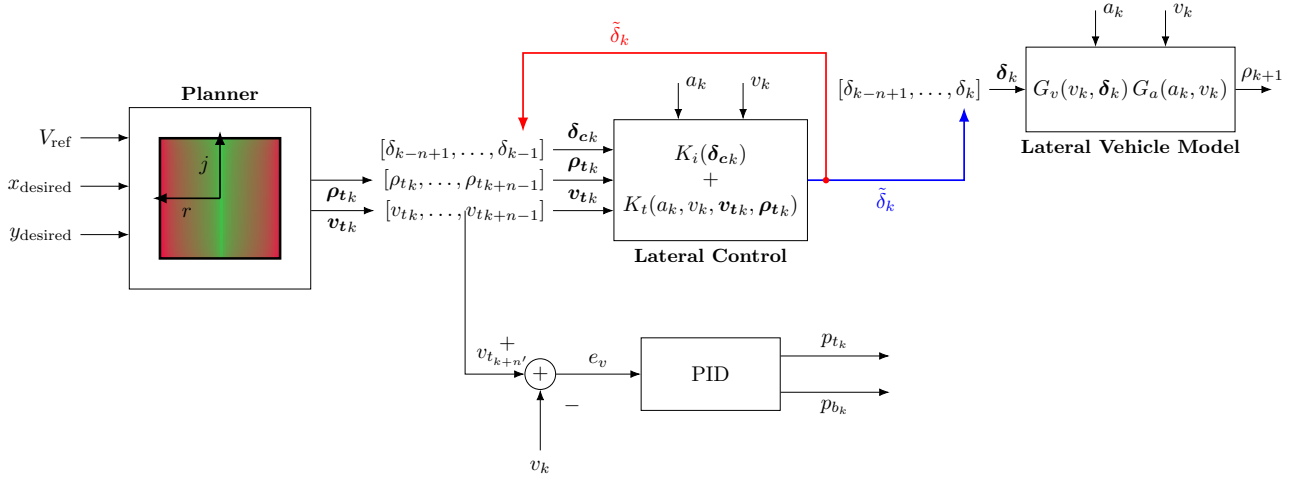


Fig. 1: Block diagram of the proposed pipeline: a minimum-jerk planner selects the optimal primitive from a jerk-based grid (green), given a lookahead target position ($x_{\text{desired}}, y_{\text{desired}}$) and reference speed computed according to the maximum road curvature (V_{ref}), and outputs future curvature ρ_{t_k} and velocity v_{t_k} profiles. A PID handles longitudinal control, while steering is generated by a neural network trained in a closed loop with a neural vehicle model. The blue line highlights the connection established between the Lateral Controller and the Model during the training phase. The red line highlights the Closed Loop that automatically performs the rolling window within the framework. Here, δ is used to represent the steer angle, whereas p_{t_k} and p_{b_k} are the pedals.

allows the motion planning layer to be implemented either through closed-form analytical solutions or via structured neural networks trained to approximate the optimal control manifold.

Regardless of the implementation, the resulting primitives serve as a basis for the decision-making layer. By evaluating a *saliency* metric against these candidates, the system measures the quality and robustness of the trajectories relative to the environmental context, ensuring the final control output is both optimal and context-aware.

More specifically, the total saliency is computed as a weighted sum of longitudinal and lateral contributions, designed to balance efficiency, comfort, and safety.

The longitudinal component consists of: (i) the maneuver duration from an initial condition to a final one; (ii) the OCP cost; and (iii) a velocity constraint penalty.

Similarly, the lateral saliency accounts for: (i) the time to reach the road boundary; (ii) the time required to re-enter the bounds; and (iii) the initial lateral jerk r_0 .

To enable the management of complex environments, an additional term derived from a structured neural network can be integrated into this calculation. This learned component allows the planner to adapt to non-linear environmental features and sophisticated constraints that are difficult to model analytically, effectively refining the decision-making process in real-time.

Following the saliency assignment, the *motor cortex* is constructed to provide a topographic organization of the motion space. In this structure, action pairs are arranged onto a two-dimensional grid, where each cell corresponds to a specific combination of longitudinal and lateral jerk, and its value represents the total saliency of that maneuver. By placing similar actions in proximity, a structured map of the available motion space is created. This organization

ensures the explainability of the planner, as the relationship between specific control inputs and their associated costs, such as those resulting from road boundaries, can be clearly visualized.

The final action selection is performed through a winner-take-all procedure, which identifies the grid cell with the minimum saliency value. This process results in the selection of the most efficient and safe trajectory from the calculated options.

D. nnodely Framework

The proposed architecture is implemented using nnodely, an open-source framework based on representing model-structured neural networks MSNNs as explicit structured representations of mathematical models. Rather than treating neural networks as opaque computation graphs, nnodely formalizes their structure, parameters, and temporal dependencies through a unified abstraction that makes architectural choices explicit and interpretable.

A key feature of nnodely is the possibility to construct complex architectures through the composition of multiple sub-models. Individual components, such as the vehicle forward model and the inverse controller, can be defined and trained independently, and later interconnected through differentiable operations. In particular, the framework provides native support for *closed loop* connections, enabling the realization of recurrent and feedback structures required for control synthesis via mental simulation. This capability is central to the proposed approach, where the inverse controller is trained by embedding it within the learned forward dynamics and optimizing the closed-loop behavior.

From a modeling perspective, nnodely provides a set of domain-aligned building blocks that directly map to the components introduced in this paper. The vehicle model is

naturally expressed through a combination of *FIR filters*, capturing temporal dynamics, and *parametric functions* encoding physically-inspired relations such as the handling diagram. Similarly, the controller structure uses the same primitives, ensuring architectural consistency between forward and inverse models. The use of *LocalModel* and *Fuzzify* operators enables the representation of nonlinear and regime-dependent behaviors through smooth interpolation across operating conditions. In the present work, recurrent training plays a crucial role, as it allows optimization over multi-step rollouts and enables the learning of control policies through differentiable simulation. Temporal coherence is preserved through dedicated data handling mechanisms, where input signals are organized into consistent time windows that match the structure of the model. This is essential for capturing the dynamic nature of the system and ensuring stability during closed-loop training.

Finally, *nnodely* provides a complete pipeline from model definition to deployment, including dataset construction, training, validation, and export. Models can be serialized into a framework-independent representation and deployed in external environments, such as the IPG CARMAKER® simulator used in this study. This end-to-end workflow ensures reproducibility and facilitates the integration of structured neural components into real-world control systems.

III. EXPERIMENTAL RESULTS

A. Current Experimental Setup

In the current implementation, the architecture adopts a hybrid approach: the lateral low-level control is learning-based, while the longitudinal control and motion planning layers maintain a deterministic structure. This configuration allows for the validation of structured neural network components within a controlled, modular framework.

While this setup provides a solid foundation for integrating learning-based methods, extending this approach to the remaining layers of the pipeline represents a key direction for future work. In particular, incorporating learning-based strategies into longitudinal control and motion planning could further enhance adaptability and overall system performance.

B. Case study

This case study evaluates the vehicle under standard operating conditions, with road curvature representative of typical highway scenarios. The range of speeds and requested maneuvers is intentionally limited, as several simplifying assumptions are introduced for this preliminary analysis. The reference speed (V_{ref} , as shown in Fig. 1) is dynamically modulated based on the maximum curvature within a future time window, while maintaining conservative safety margins. Additionally, the lateral controller has been trained to handle higher acceleration demands, ensuring adequate responsiveness under these conditions. Overall, this case study demonstrates the full pipeline operating cohesively and provides a foundation for discussing future developments and potential extensions of the approach.



Fig. 2: IPG CARMAKER® simulation

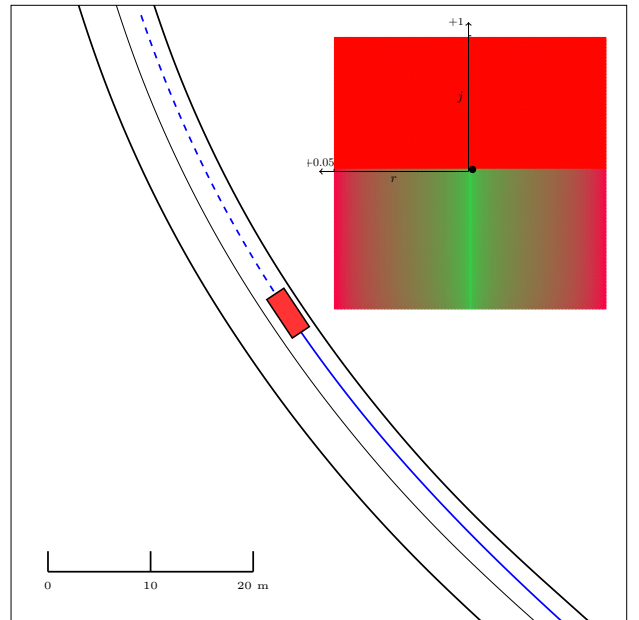
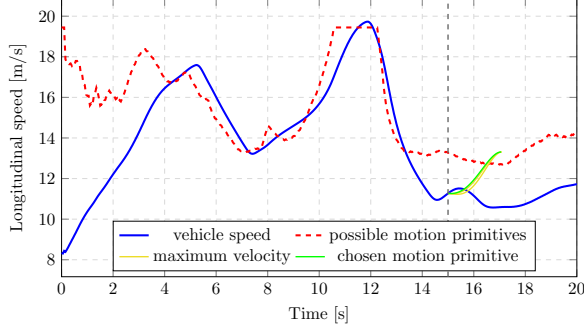


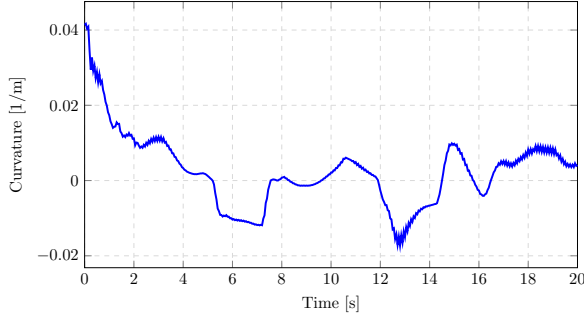
Fig. 3: Realized vehicle trajectory and visualization of action selection in the motor cortex.

Figure 3 illustrates the fundamental operating principle of the proposed planner. Even in this simplified scenario, several relevant observations can be made. In the longitudinal domain, the regions associated with positive acceleration are significantly inhibited as the vehicle approaches its maximum permitted speed. This behaviour derives from the formulation of the salience, which heavily penalises actions that would lead to exceeding the reference speed, while relatively lower penalties are assigned to negative acceleration (*i.e.*, deceleration). In the lateral domain, the salience distribution shows a pronounced band of high values around the central region. This can be interpreted as a consequence of the vehicle already being engaged in a cornering manoeuvre and operating under conditions close to steady state, where lateral acceleration remains approximately constant.

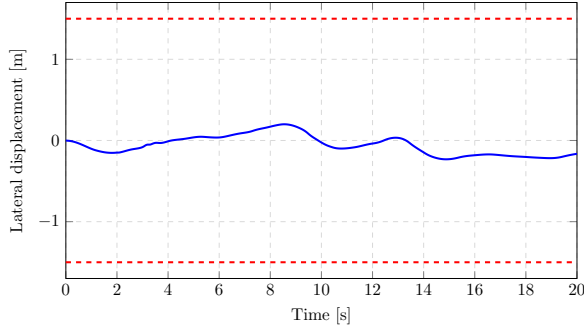
Fig. 4a illustrates the framework's ability to maintain specific velocity limits while selecting the most efficient



(a). Comparison between target longitudinal velocity and achieved value. For time $T=15s$, some of the possible motion primitives are displayed, alongside the chosen one in the action selection process.



(b). Vehicle curvature.



(c). Vehicle lateral displacement, where the dashed red lines represent the bounds within which the vehicle bounding box remains inside the lane.

Fig. 4: Simulation results

motion primitive from the motor cortex. The results show that the planner identifies the maneuver that completes the task in the shortest time while minimizing control costs (represented by the green curve). It is worth noting that the target speed is defined at the current instant and may differ from the speed required upon completion of the manoeuvre. However, the adoption of a moving-horizon strategy ensures that the target is continuously updated. This iterative process guarantees that any discrepancy between instantaneous and future speed requirements does not adversely affect the overall performance or stability of the planner. Finally, Fig. 4c confirms that the vehicle remains within the prescribed road limits throughout the manoeuvre, thus demonstrating that lateral constraints are effectively incorporated into the planning

framework. Fig. 4b shows the curvature profile generated by the vehicle. The observed oscillations are mainly attributable to inconsistencies arising from the continuous updating of the lateral motion primitives, which can vary significantly from one iteration to the next, thereby compromising the performance of the lateral controller. Two main factors contribute to this behaviour. Firstly, the planner ideally requires the reconstruction of the road centreline in terms of clothoid segments. However, in this case study, the reference clothoid is recalculated at every cycle to directly link the vehicle's current state to the destination point. This results in discontinuities in the planned curvature profile. Secondly, the use of fixed planning parameters, such as the look-ahead distance, introduces further limitations, as these parameters should inherently depend on the vehicle's speed. One potential solution involves simultaneously training both the controller parameters and planner parameters within a closed-loop connection. In this configuration, additional penalties—such as those on the steering angle derivative—can be introduced to enforce smoother control actions. Furthermore, key planning parameters (*i.e.*, lookahead distance) could be updated adaptively via a neural network, enabling greater robustness under variable operating conditions. From a computational perspective, the neural network could also be used to dynamically adjust the number of motion primitives and the grid resolution, concentrating computational resources in the most relevant regions of the action space.

IV. CONCLUSIONS AND FUTURE WORK

This work presented a first experimental validation of a hybrid framework for autonomous driving based on Model Structured Neural Networks, focusing on the integration of motion planning and low-level control within a differentiable architecture. The proposed approach has been evaluated in a high-fidelity commercial simulation environment (IPG CAR-MAKER®), where the planner and the control components are designed to be implemented through structured neural networks. At the current stage, only the low-level control module has been fully implemented using the nnodely framework, demonstrating the feasibility of learning structured neural controllers and deploying them in closed-loop simulations. The planner component, although conceptually integrated within the proposed architecture, remains to be fully developed within the same framework. Future work will focus on extending the nnodely-based implementation to the entire control pipeline, including the motion planning layer. This will enable the realization of a fully unified and differentiable architecture, where all modules are trained within a common framework. Such an approach would allow multi-level training strategies, supporting both independent module optimization and end-to-end fine-tuning across planning and control. Another key research direction concerns online adaptation. Since the proposed controllers are based on neural networks, they naturally enable the possibility of adapting their behavior during operation. Investigating robust and safe online learning mechanisms will be essential

to further improve generalization capabilities and allow the system to cope with previously unseen conditions.

Overall, this work represents an initial step toward structured, learnable, and adaptable autonomous driving systems, highlighting the potential of MSNN-based approaches for bridging modular and end-to-end paradigms.

REFERENCES

- [1] J. Betz, H. Zheng, A. Liniger, U. Rosolia, P. Karle, M. Behl, V. Krovi, and R. Mangharam, "Autonomous vehicles on the edge: A survey on autonomous vehicle racing," *IEEE Open Journal of Intelligent Transportation Systems*, vol. 3, pp. 458–488, 2022.
- [2] J. Zhao, W. Zhao, B. Deng, Z. Wang, F. Zhang, W. Zheng, W. Cao, J. Nan, Y. Lian, and A. F. Burke, "Autonomous driving system: A comprehensive survey," *Expert Systems with Applications*, vol. 242, p. 122836, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0957417423033389>
- [3] B. Paden, M. Čáp, S. Z. Yong, D. Yershov, and E. Frazzoli, "A survey of motion planning and control techniques for self-driving urban vehicles," *IEEE Transactions on intelligent vehicles*, vol. 1, no. 1, pp. 33–55, 2016.
- [4] L. Chen, P. Wu, K. Chitta, B. Jaeger, A. Geiger, and H. Li, "End-to-end autonomous driving: Challenges and frontiers," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 12, pp. 10 164–10 183, 2024.
- [5] J. Hu, Y. Wang, S. Cheng, J. Xu, N. Wang, B. Fu, Z. Ning, J. Li, H. Chen, C. Feng *et al.*, "A survey of decision-making and planning methods for self-driving vehicles," *Frontiers in Neurobotics*, vol. 19, p. 1451923, 2025.
- [6] M. Da Lio, D. Bortoluzzi, and G. P. Rosati Papini, "Modelling longitudinal vehicle dynamics with neural networks," *Vehicle System Dynamics*, vol. 58, no. 11, pp. 1675–1693, 2020. [Online]. Available: <https://doi.org/10.1080/00423114.2019.1638947>
- [7] M. Piccinini, S. Taddei, M. Larcher, M. Piazza, and F. Biral, "A physics-driven artificial agent for online time-optimal vehicle motion planning and control," *IEEE Access*, vol. 11, pp. 46 344–46 372, 2023.
- [8] M. Da Lio, R. Donà, G. P. Rosati Papini, F. Biral, and H. Svensson, "A mental simulation approach for learning neural-network predictive control (in self-driving cars)," *IEEE Access*, vol. 8, pp. 192 041–192 064, 2020.
- [9] M. Werling, J. Ziegler, S. Kammel, and S. Thrun, "Optimal trajectory generation for dynamic street scenarios in a frenet frame," in *2010 IEEE international conference on robotics and automation*. IEEE, 2010, pp. 987–993.
- [10] R. Trauth, K. Moller, G. Würsching, and J. Betz, "Frenetix: A high-performance and modular motion planning framework for autonomous driving," *IEEE Access*, 2024.
- [11] M. Piazza, M. Piccinini, S. Taddei, and F. Biral, "Mptree: A sampling-based vehicle motion planner for real-time obstacle avoidance," *IFAC-PapersOnLine*, vol. 58, no. 10, pp. 146–153, 2024.